

Reviewer Pack — Minimal Rank of Geometric Langlands Duality over XOR-AND Tre...

Entry 9b6defe8501b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 01:24:10 UTC

Minimal Rank of Geometric Langlands Duality over XOR-AND Tree Width

Entry ID: 9b6defe8501b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 01:24:10 UTC

1. Conjecture

Field A (mathematical branch): Geometric Langlands Program **Field B** (complexity object): XOR-AND Tree Width

Statement:

[‘For each XOR-AND tree T with n leaves, there exists a finite field extension K and an object X in the geometric Langlands correspondence associated with T such that the minimal rank of the moduli space of K -points on X is $\Theta(\omega(T))$, where $\omega(T)$ denotes the width of T .’, ‘For all instances T with width ≤ 40 and depth ≥ 3 , if $\omega(T) < n/2$, then the above statement holds with a constant $c > 0$.’]

Rationale (proposer’s reasoning):

[‘The Geometric Langlands Program provides a bridge between algebraic geometry and the representation theory of Lie groups. This conjecture leverages this bridge to relate geometric properties to computational complexity via XOR-AND tree width, an object with known connections to NP-hard problems.’, ‘If true, it would offer a new perspective on understanding the complexity of problems defined by XOR-AND trees by invoking deep algebraic-geometric structures.’]

Taxonomy category: Geometric_Langlands (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3047f2ebc9cddbdf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 24 out of 30 random seeds, the ratio of the computed minimal rank to $\omega(T)$ is greater than or equal to 0.5 and less than or equal to 1.5, where $\omega(T)$ is the tree width of XOR-AND trees with n leaves ≤ 40 and depth ≥ 3 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Geometric Langlands Program" AND "XOR-AND tree width" - "minimal rank moduli space" IN "geometric Langlands correspondence" AND "tree width" ≤ 40 - "finite field extension" AND "object X" IN "geometric Langlands" AND width $< n/2$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_xor_and_tree(n, depth):
    if n == 1:
        return [random.choice([0, 1])]
    elif depth == 1:
        return [random.choice([0, 1]) for _ in range(n)]

    left = generate_xor_and_tree(n // 2, depth - 1)
    right = generate_xor_and_tree(n - n // 2, depth - 1)
    return [left[i] ^ right[i % len(right)] for i in range(n)]

def compute_minimal_rank(tree):
    if not tree:
        return 0
    elif len(tree) == 1:
        return 1

    left_rank = compute_minimal_rank([t[0] for t in tree])
    right_rank = compute_minimal_rank([t[1] for t in tree])
    return max(left_rank, right_rank) + 1

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    depth = random.randint(3, 3)
    tree = generate_xor_and_tree(n, depth)
```

```

omega_T = len(tree)

minimal_rank = compute_minimal_rank(tree)
ratio = Fraction(minimal_rank, omega_T)

return {
    "metric_name": "minimal_rank_ratio",
    "metric_value": float(ratio),
    "instances_tested": 1,
    "conjecture_holds": 0.5 <= ratio <= 1.5,
    "counterexample": "" if 0.5 <= ratio <= 1.5 else f"Ratio {ratio} out of bounds"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(3, 6)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if 0.5 <= r["metric_value"] <= 1.5) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_support_fraction support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_915dcf06.py", line 63, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_915dcf06.py", line 43, in run_trial
    tree = generate_xor_and_tree(n, depth)
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_915dcf06.py", line 24, in generate_xor_and_tree
    left = generate_xor_and_tree(n // 2, depth - 1)
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_915dcf06.py", line 26, in generate_xor_and_tree
    return [left[i] ^ right[i % len(right)] for i in range(n)]
           ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to provide evidence for the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing. If the support condition is met after addressing the crash issue, then the verdict can be upgraded to SUPPORTED.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11722
2	preregistration	ollama_remote	glm4:latest	0	0	6172
3	novelty	ollama_remote	glm4:latest	0	0	4878
4	novelty	ollama_remote	glm4:latest	0	0	5590
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13862
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9395
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8437
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8462
9	judge	ollama_remote	glm4:latest	0	0	9526

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 78043 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/9b6defe8501b.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/9b6defe8501b.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/9b6defe8501b.tar.gz (if generated)